

LAB DAY 2

NAME: AGILAN VIYASH

REG. NO.: 192511346

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

FIRST PALINDROMIC STRING

PYTHON CODE:

```
def first_palindrome(words):  
    for word in words:  
        if word == word[::-1]:  
            return word  
    return ""
```

Example 1

```
words = ["abc", "car", "ada", "racecar", "cool"]  
print(first_palindrome(words))
```

Example 2

```
words = ["notapalindrome", "racecar"]  
print(first_palindrome(words))
```

EXECUTION:

```
CC.py - C:/Users/Agilan/OneDrive/Documents/CC.py (3.14.2)
File Edit Format Run Options Window Help
def first_palindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

# Example 1
words = ["abc", "car", "ada", "racecar", "cool"]
print(first_palindrome(words))

# Example 2
words = ["notapalindrome", "racecar"]
print(first_palindrome(words))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/CC.py =====
ada
racecar
>>> |
```

COUNT COMMON ELEMENTS IN TWO ARRAYS

PYTHON CODE:

```
def find_intersection_values(nums1, nums2):

    set1 = set(nums1)

    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)

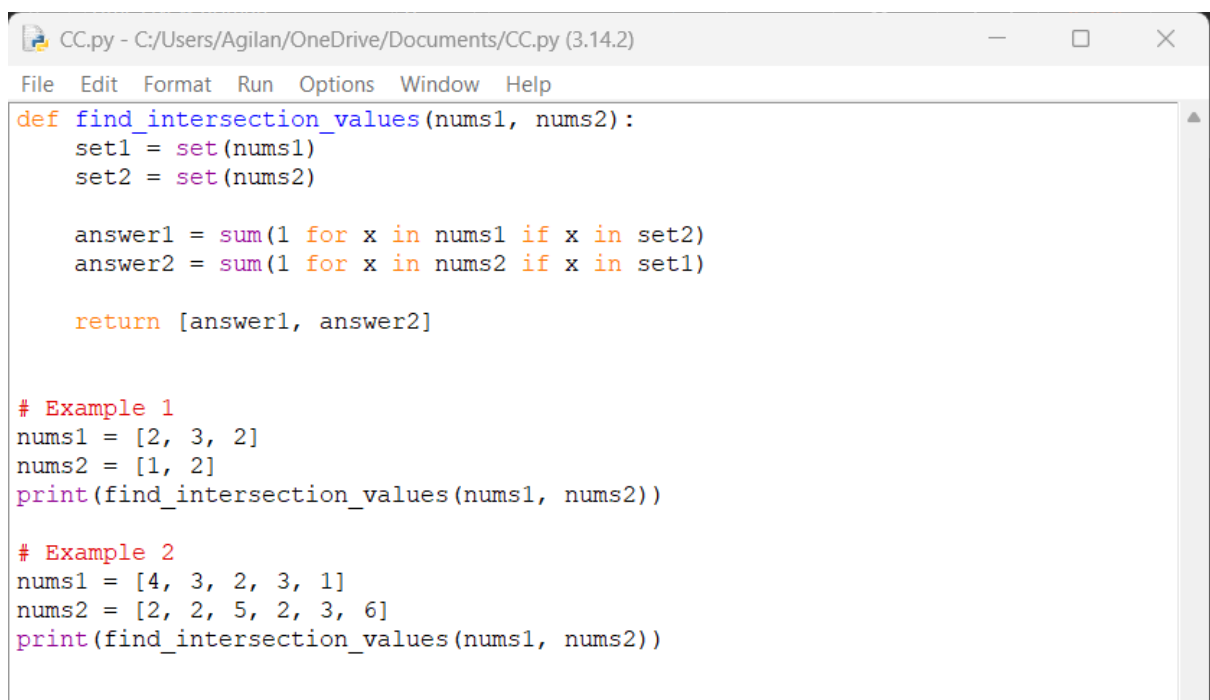
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]
```

```
# Example 1
nums1 = [2, 3, 2]
nums2 = [1, 2]
print(find_intersection_values(nums1, nums2))
```

```
# Example 2
nums1 = [4, 3, 2, 3, 1]
nums2 = [2, 2, 5, 2, 3, 6]
print(find_intersection_values(nums1, nums2))
```

EXECUTION:

A screenshot of a Python IDE window titled "CC.py - C:/Users/Agilan/OneDrive/Documents/CC.py (3.14.2)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor displays the following Python code:

```
def find_intersection_values(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]

# Example 1
nums1 = [2, 3, 2]
nums2 = [1, 2]
print(find_intersection_values(nums1, nums2))

# Example 2
nums1 = [4, 3, 2, 3, 1]
nums2 = [2, 2, 5, 2, 3, 6]
print(find_intersection_values(nums1, nums2))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/CC.py =====
[2, 1]
[3, 4]
>>>
```

SUM OF SQUARES OF DISTINCT COUNTS OF ALL SUBARRAYS

PYTHON CODE:

```
def sum_of_distinct_squares(nums):
```

```
    total = 0
```

```
    n = len(nums)
```

```
    for i in range(n):
```

```
        distinct = set()
```

```
        for j in range(i, n):
```

```
            distinct.add(nums[j])
```

```
            count = len(distinct)
```

```
            total += count * count
```

```
    return total
```

```
# Example 1
```

```
nums = [1, 2, 1]
```

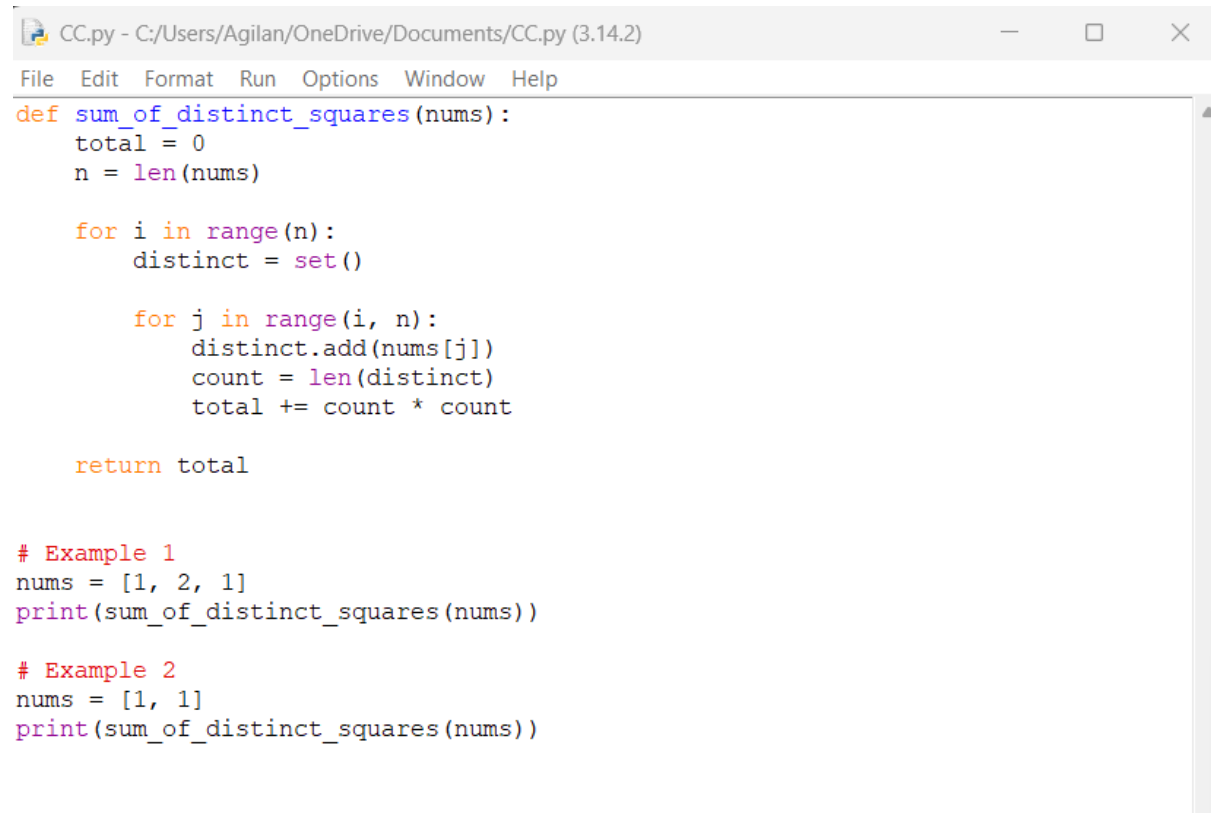
```
print(sum_of_distinct_squares(nums))
```

Example 2

nums = [1, 1]

print(sum_of_distinct_squares(nums))

EXECUTION:



```
CC.py - C:/Users/Agilan/OneDrive/Documents/CC.py (3.14.2)
File Edit Format Run Options Window Help
def sum_of_distinct_squares(nums):
    total = 0
    n = len(nums)

    for i in range(n):
        distinct = set()

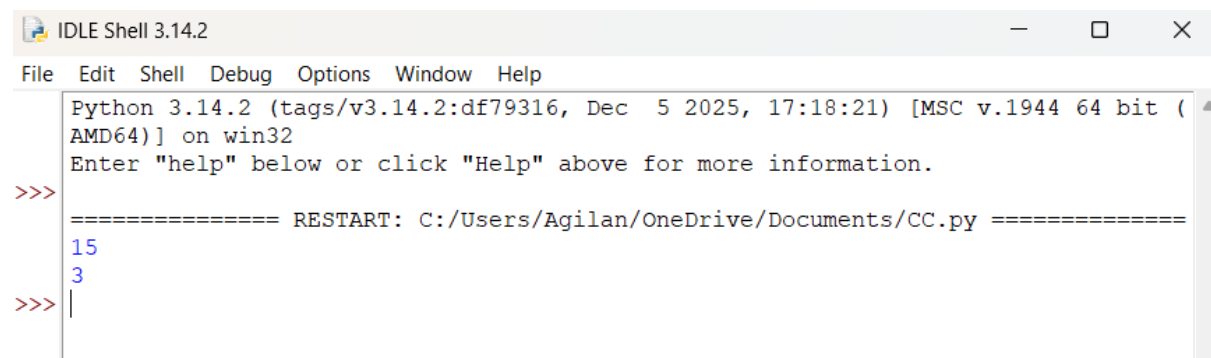
        for j in range(i, n):
            distinct.add(nums[j])
            count = len(distinct)
            total += count * count

    return total

# Example 1
nums = [1, 2, 1]
print(sum_of_distinct_squares(nums))

# Example 2
nums = [1, 1]
print(sum_of_distinct_squares(nums))
```

OUTPUT:



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/CC.py =====
15
3
>>> |
```

COUNT EQUAL-VALU PAIRS WHERE (i*j) IS DVISIBLE BY k

PYTHON CODE:

from collections import defaultdict

```
def count_pairs(nums, k):
    groups = defaultdict(list)

    # Store indices for each value
    for i, value in enumerate(nums):
        groups[value].append(i)

    count = 0

    # Check pairs having the same value
    for indices in groups.values():
        for x in range(len(indices)):
            for y in range(x + 1, len(indices)):
                i = indices[x]
                j = indices[y]

                if (i * j) % k == 0:
                    count += 1

    return count
```

Example 1

```
nums = [3, 1, 2, 2, 2, 1, 3]
```

```
k = 2
```

```
print(count_pairs(nums, k))
```

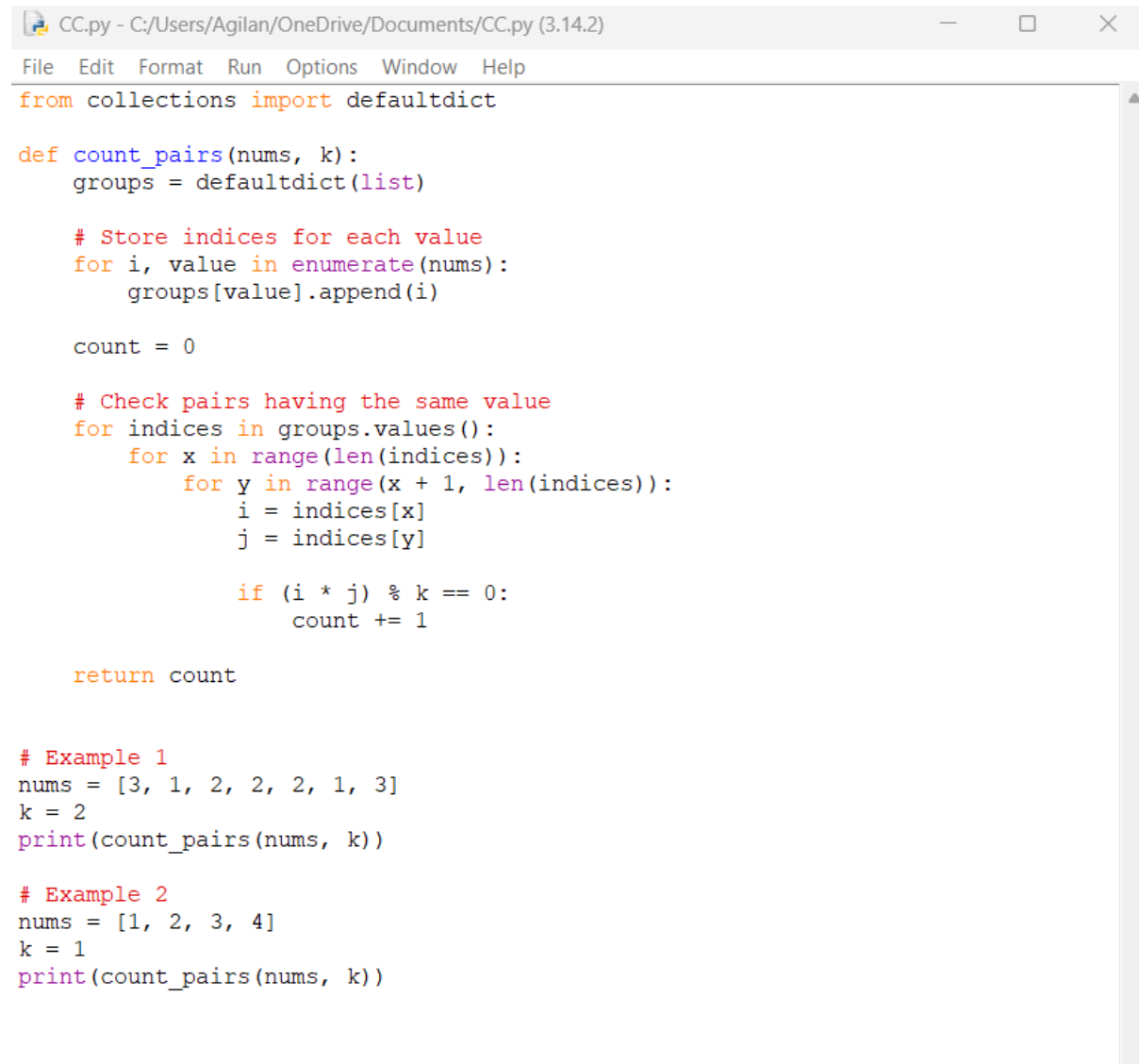
Example 2

```
nums = [1, 2, 3, 4]
```

```
k = 1
```

```
print(count_pairs(nums, k))
```

EXECUTION:

A screenshot of a Python IDE window titled "CC.py - C:/Users/Agilan/OneDrive/Documents/CC.py (3.14.2)". The window contains a Python script with a function `count_pairs` and two example calls. The script uses `collections.defaultdict` to group indices by value and then checks pairs within each group. The first example uses `nums = [3, 1, 2, 2, 2, 1, 3]` and `k = 2`, and the second example uses `nums = [1, 2, 3, 4]` and `k = 1`.

```
CC.py - C:/Users/Agilan/OneDrive/Documents/CC.py (3.14.2)
File Edit Format Run Options Window Help
from collections import defaultdict

def count_pairs(nums, k):
    groups = defaultdict(list)

    # Store indices for each value
    for i, value in enumerate(nums):
        groups[value].append(i)

    count = 0

    # Check pairs having the same value
    for indices in groups.values():
        for x in range(len(indices)):
            for y in range(x + 1, len(indices)):
                i = indices[x]
                j = indices[y]

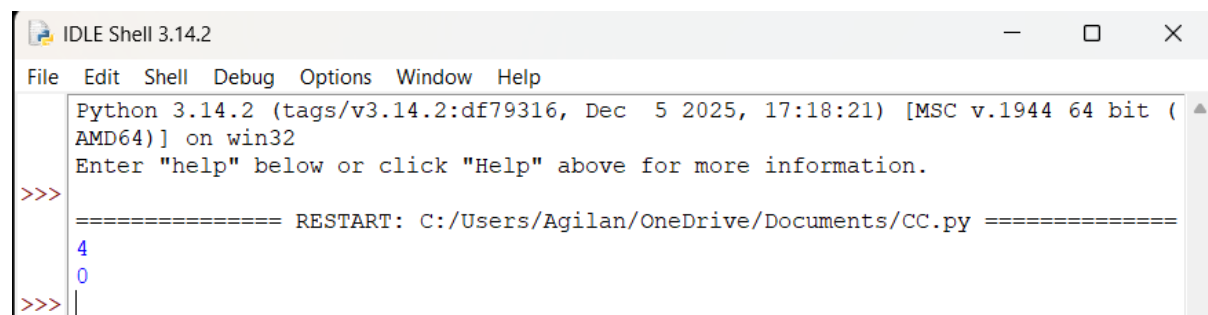
                if (i * j) % k == 0:
                    count += 1

    return count

# Example 1
nums = [3, 1, 2, 2, 2, 1, 3]
k = 2
print(count_pairs(nums, k))

# Example 2
nums = [1, 2, 3, 4]
k = 1
print(count_pairs(nums, k))
```

OUTPUT:

A screenshot of an IDLE Shell window titled "IDLE Shell 3.14.2". The window shows the output of the script execution. It displays the Python version and architecture, followed by a restart message and the output of the two example calls: 4 and 0.

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/CC.py =====
4
0
>>> |
```

FIND THE MAXIMUM ELEMENT IN AN ARRAY

PYTHON CODE:

```
def find_maximum(nums):  
    if not nums:  
        return None  
  
    maximum = nums[0]  
  
    for num in nums:  
        if num > maximum:  
            maximum = num  
  
    return maximum
```

Test Case 1

```
nums = [1, 2, 3, 4, 5]  
print(find_maximum(nums))
```

Test Case 2

```
nums = [7, 7, 7, 7, 7]  
print(find_maximum(nums))
```

Test Case 3

```
nums = [-10, 2, 3, -4, 5]  
print(find_maximum(nums))
```

EXECUTION:


```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def find_maximum(nums):
    if not nums:
        return None

    maximum = nums[0]

    for num in nums:
        if num > maximum:
            maximum = num

    return maximum

# Test Case 1
nums = [1, 2, 3, 4, 5]
print(find_maximum(nums))

# Test Case 2
nums = [7, 7, 7, 7, 7]
print(find_maximum(nums))

# Test Case 3
nums = [-10, 2, 3, -4, 5]
print(find_maximum(nums))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
5
7
5
>>> |
```

SORT THE LIST AND FIND THE MAXIMUM ELEMENT

PYTHON CODE:

```
def find_maximum(nums):

    if not nums:

        return None

    # Sort the list using Merge Sort
```

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

sorted_nums = merge_sort(nums)

# Maximum element is the last element

```

```
return sorted_nums[-1]
```

```
# Test Case 1: Empty List
```

```
nums = []
```

```
print(find_maximum(nums))
```

```
# Test Case 2: Single Element List
```

```
nums = [5]
```

```
print(find_maximum(nums))
```

```
# Test Case 3: All Elements are the Same
```

```
nums = [3, 3, 3, 3, 3]
```

```
print(find_maximum(nums))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

sorted_nums = merge_sort(nums)

# Maximum element is the last element
return sorted_nums[-1]

# Test Case 1: Empty List
nums = []
print(find_maximum(nums))

# Test Case 2: Single Element List
nums = [5]
print(find_maximum(nums))

# Test Case 3: All Elements are the Same
nums = [3, 3, 3, 3, 3]
print(find_maximum(nums))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
None
5
3
>>>
```

REMOVE DUPLICATE ELEMENT FROM A LIST

PYTHON CODE:

```
def unique_elements(nums):
```

```
    unique = []
```

```
    seen = set()
```

```
    for num in nums:
```

```
        if num not in seen:
```

```
            seen.add(num)
```

```
            unique.append(num)
```

```
    return unique
```

```
# Test Case 1: Some Duplicate Elements
```

```
nums = [3, 7, 3, 5, 2, 5, 9, 2]
```

```
print(unique_elements(nums))
```

```
# Test Case 2: Negative and Positive Numbers
```

```
nums = [-1, 2, -1, 3, 2, -2]
```

```
print(unique_elements(nums))
```

```
# Test Case 3: List with Large Numbers
```

```
nums = [1000000, 999999, 1000000]
```

```
print(unique_elements(nums))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def unique_elements(nums):
    unique = []
    seen = set()

    for num in nums:
        if num not in seen:
            seen.add(num)
            unique.append(num)

    return unique

# Test Case 1: Some Duplicate Elements
nums = [3, 7, 3, 5, 2, 5, 9, 2]
print(unique_elements(nums))

# Test Case 2: Negative and Positive Numbers
nums = [-1, 2, -1, 3, 2, -2]
print(unique_elements(nums))

# Test Case 3: List with Large Numbers
nums = [1000000, 999999, 1000000]
print(unique_elements(nums))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
>>>
```

TOWER OF HANOI

PYTHON CODE:

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
swapped = False
```

```
for j in range(0, n - i - 1):
```

```
    if arr[j] > arr[j + 1]:
```

```
        arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
        swapped = True
```

```
# Stop if the array is already sorted
```

```
if not swapped:
```

```
    break
```

```
return arr
```

```
# Test Case
```

```
arr = [64, 34, 25, 12, 22, 11, 90]
```

```
print("Original array:", arr)
```

```
print("Sorted array:", bubble_sort(arr))
```

```
EXECUTION:
```

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help

def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Stop if the array is already sorted
        if not swapped:
            break

    return arr

# Test Case
arr = [64, 34, 25, 12, 22, 11, 90]

print("Original array:", arr)
print("Sorted array:", bubble_sort(arr))
```

OUTPUT:

BINARY SEARCH

PYTHON CODE:

```
def binary_search(arr, key):
```

```
    # Sort the array first
```

```
    arr.sort()
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```



```
if arr[mid] == key:
```

```
    return mid
```

```
elif arr[mid] < key:
```

```
    low = mid + 1
```

```
else:
```

```
    high = mid - 1
```

```
return -1
```

```
# Test Case 1
```

```
X = [3, 4, 6, -9, 10, 8, 9, 30]
```

```
KEY = 10
```

```
position = binary_search(X, KEY)
```

```
if position != -1:
```

```
    print("Element", KEY, "is found at position", position + 1)
```

```
else:
```

```
    print("Element", KEY, "is not found")
```

```
# Test Case 2
```

```
X = [3, 4, 6, -9, 10, 8, 9, 30]
```

```
KEY = 100
```

```
position = binary_search(X, KEY)
```

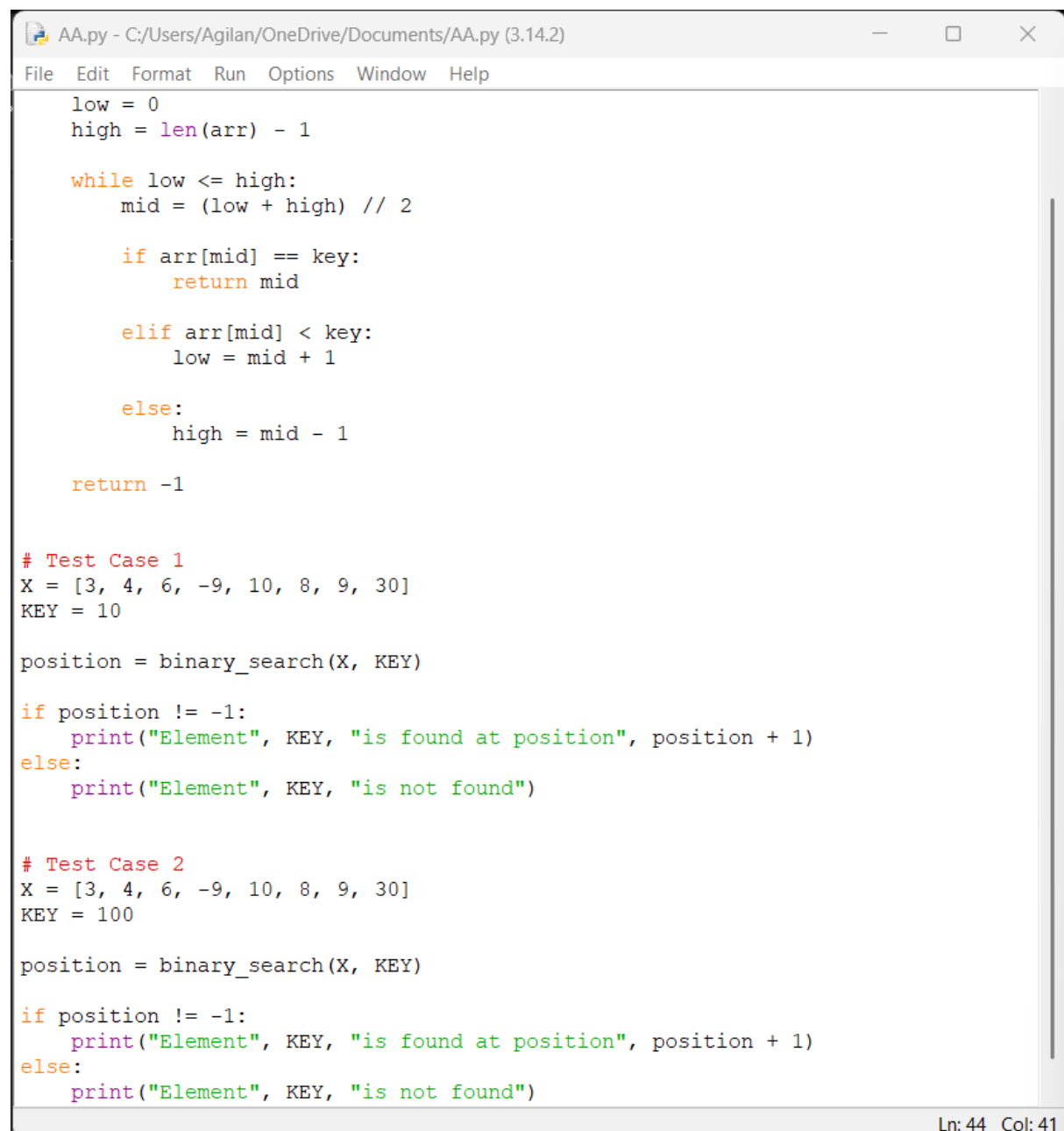
```
if position != -1:
```

```
    print("Element", KEY, "is found at position", position + 1)
```

```
else:
```

```
    print("Element", KEY, "is not found")
```

EXECUTION:



```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help

low = 0
high = len(arr) - 1

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        return mid

    elif arr[mid] < key:
        low = mid + 1

    else:
        high = mid - 1

return -1

# Test Case 1
X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 10

position = binary_search(X, KEY)

if position != -1:
    print("Element", KEY, "is found at position", position + 1)
else:
    print("Element", KEY, "is not found")

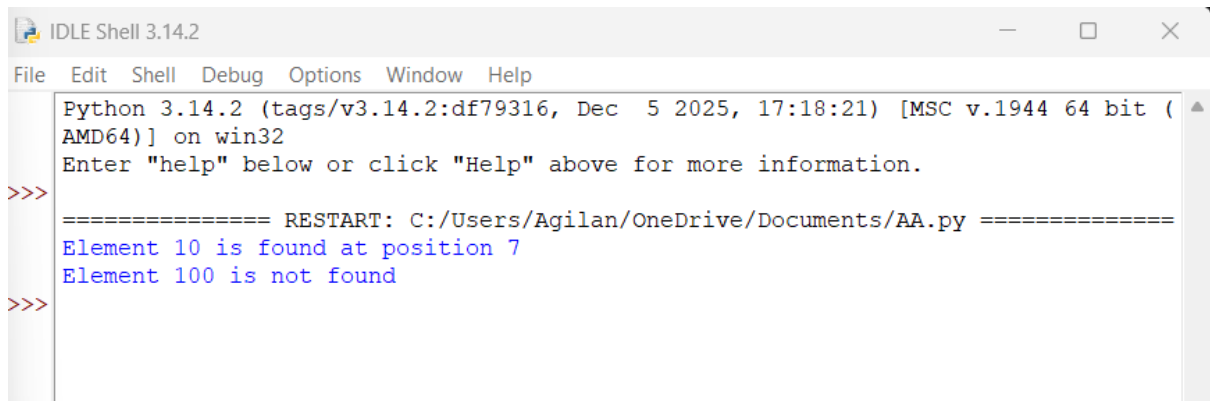
# Test Case 2
X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 100

position = binary_search(X, KEY)

if position != -1:
    print("Element", KEY, "is found at position", position + 1)
else:
    print("Element", KEY, "is not found")

Ln: 44 Col: 41
```

OUTPUT:



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Element 10 is found at position 7
Element 100 is not found
>>>
```

INSERTION SORT

PYTHON CODE:

```
def merge_sort(nums):
```

```
    if len(nums) <= 1:
```

```
        return nums
```

```
    mid = len(nums) // 2
```

```
    left = merge_sort(nums[:mid])
```

```
    right = merge_sort(nums[mid:])
```

```
    result = []
```

```
    i = 0
```

```
    j = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] <= right[j]:
```

```
            result.append(left[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right[j])
```

```
j += 1
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
# Test Case
```

```
nums = [5, 2, 3, 1]
```

```
print("Original array:", nums)
```

```
print("Sorted array:", merge_sort(nums))
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help
def merge_sort(nums):
    if len(nums) <= 1:
        return nums

    mid = len(nums) // 2

    left = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Test Case
nums = [5, 2, 3, 1]

print("Original array:", nums)
print("Sorted array:", merge_sort(nums))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Original array: [5, 2, 3, 1]
Sorted array: [1, 2, 3, 5]
>>>
```